

Rapport de veille

1. Intro
2. Résumé rapide des deux protocoles
3. Benchmark des deux protocoles
4. Eco-conception
5. RGPD
6. Choix entre les deux protocoles

SignalWatch est une startup spécialisée dans le monitoring de capteurs industriels elle collecte jusqu'à 10 000 événements par minute (température, pression, vibration) et doit faire un choix structurant : quel protocole de communication adopter entre ses microservices ?

Deux protocoles s'affrontent aujourd'hui dans les architectures modernes : REST, largement maîtrisé et gRPC plus récent, conçu par Google pour les communications inter-services haute performance.

Objectif de cette veille

Ce document a pour but de produire une recommandation argumentée et chiffrée de sorte à justifier le choix entre les deux protocoles.

Pour ça, nous avons combiné deux approches complémentaires : Une veille documentaire sur les protocoles existants (REST, gRPC, GraphQL, brokers de messages)

Ainsi qu'un benchmark empirique avec deux micro-services identiques implémentés et soumis aux mêmes scénarios de charge, pour obtenir des données réelles plutôt qu'une comparaison purement théorique.

Résumé des deux protocoles ([source](#))

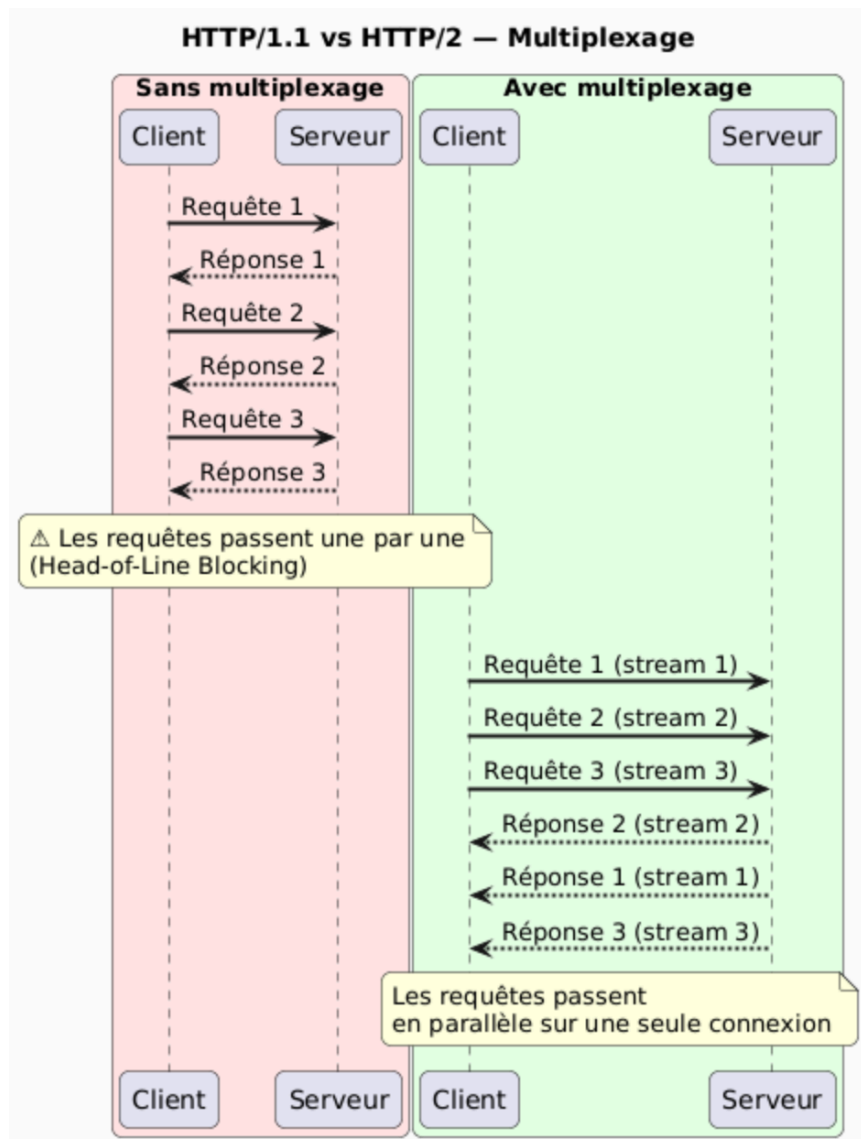
gRPC

gRPC a comme base le protocole RPC (Remote Procedure Call), c'est une version "améliorée" par Google et ça repose sur deux technologies : le HTTP 2 et le Protocol Buffers.

Le HTTP/2 permet le multiplexage et le Protobuf est un format de sérialisation, contrairement au JSON de REST les données sont encodées en binaire. Ça les rends plus compact et plus rapides à parser. L'inconvénient est la couche de complexité que ça peut ajouter.

Multiplexage

Le multiplexage permet de gérer plusieurs flux de requêtes à la fois (fonctionne pour expédition et aussi réception de requêtes). Voici un schéma pour illustrer :



Avec cette méthode, si un stream est ralenti il n'affectera pas les autres, alors que sans multiplexage même si les requêtes sont rapides il suffit qu'une seule soit lente et ça ralentit celles qui sont derrière.

ProtoBuf

C'est un format qui sérialise les données en binaire et au lieu des noms de champs (ex : "email:") Protobuf remplace ça par des numéros qui sont définis dans des fichiers .proto.

Ces fichiers servent aussi de contrat d'interface pour mettre l'expéditeur et le destinataire d'accord :

```
syntax = "proto3";

message Customer {
    required int32 id = 1;
    required string name = 2;
    required string email = 3;
    optional string address = 4;
}
```

Avec les fichiers.proto on peut ensuite utiliser la commande suivant pour générer nos classes à partir de proto (ça génère coté serveur et coté client également).

Les fichiers proto font office de source de vérité unique et si on change une classe coté serveur sans regénéré coté client la communication casse. Il faut donc mettre à jour en générant la classe sur les des parties (serveur et client).

Exemple de génération pour du Go, on indique le langage souhaité et le path (soit path vers serveur soit vers client):

```
bash
# Côté serveur
protoc --go_out=./server/src sensor.proto

# Côté client
protoc --go_out=./client/src sensor.proto
```

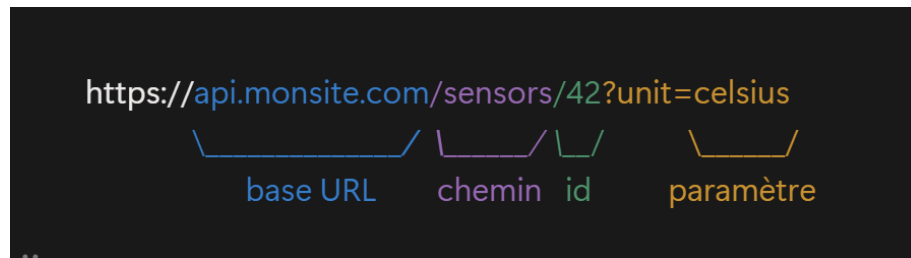
REST

REST (Representational State Transfer) c'est un style d'architecture basé sur le protocole HTTP/1.1 (et compatible HTTP/2). Contrairement à gRPC qui est un framework avec des contraintes techniques précises, REST est un ensemble de conventions qui repose sur deux éléments centraux : les méthodes HTTP et le format JSON.

Les méthodes HTTP (GET, POST, PUT, DELETE) définissent l'action à effectuer sur une ressource identifiée par une URL. Les données échangées sont sérialisées en JSON : un

format texte lisible par un humain, ce qui le rend facile à déboguer et à intégrer.
L'inconvénient est que JSON est plus verbeux et plus lent à parser que du binaire.

On expose chaque requête via une URL et on la manipule avec les méthodes HTTP standard GET, POST etc...



REST fonctionne sur un modèle strictement requête → réponse : le client envoie une requête, le serveur répond, la connexion se ferme (en HTTP/1.1). Chaque requête est indépendante et sans état (stateless), ce qui simplifie la scalabilité mais implique que toutes les informations nécessaires doivent être présentes dans chaque requête (token d'auth, paramètres, etc.).

En HTTP/1.1, il n'y a pas de multiplexage : les requêtes sont traitées séquentiellement sur une connexion. Une requête lente bloque les suivantes (head-of-line blocking). HTTP/2 peut être utilisé avec REST pour atténuer ce problème, mais c'est rarement configuré au niveau applicatif.

Objectif du benchmark

L'objectif de ce benchmark est de comparer le protocole REST au protocole gRPC, comme on a pu le voir dans le chapitre précédent les deux protocoles ont des spécificités communes et d'autres totalement différentes. Il est donc intéressant de comparer les performances des deux en effectuant les tests suivants :

- Le premier consiste à mesurer les performances d'une lecture unitaire, avec 1000 requêtes et 10 connexions concurrentes, afin de comparer la latence des deux protocoles.
- Le second scénario concerne l'écriture de données, avec 500 requêtes et 5 connexions concurrentes, dans le but d'évaluer le coût d'une opération de création.
- Le troisième scénario repose sur une montée en charge progressive, de 10 à 100 connexions concurrentes, pour observer la réaction des services sous une pression croissante.

Les résultats sont ensuite analysés à partir de plusieurs métriques : la latence p50, p95 et p99, le throughput (req/s), la taille moyenne des payloads et le taux d'erreur.

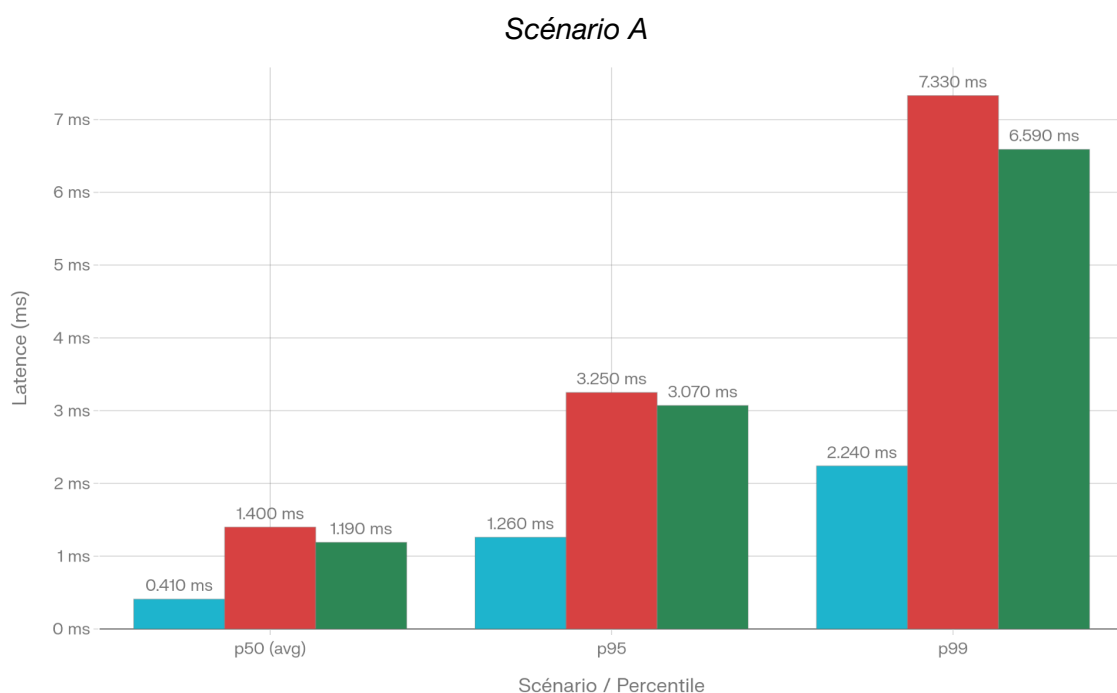
Les latences “p...” signifient le pourcentage de latences. 50% des requêtes sont inférieur ou égal à une valeur, 95% sont en dessous de cette valeur et 99% sont en dessous de cette valeur. Ca permet de s'imaginer des scénarios dans différents contextes de charge.

Les résultats montrent d'abord un avantage clair de gRPC sur la compacité des données (capacité à compacter les données d'une requête). Pour un même capteur, le payload JSON mesuré est de 230 octets contre 135 octets en Protobuf, soit un ratio d'environ 1,70. Cet écart nous confirme l'intérêt de gRPC dans une logique de réduction du volume des données échangées .

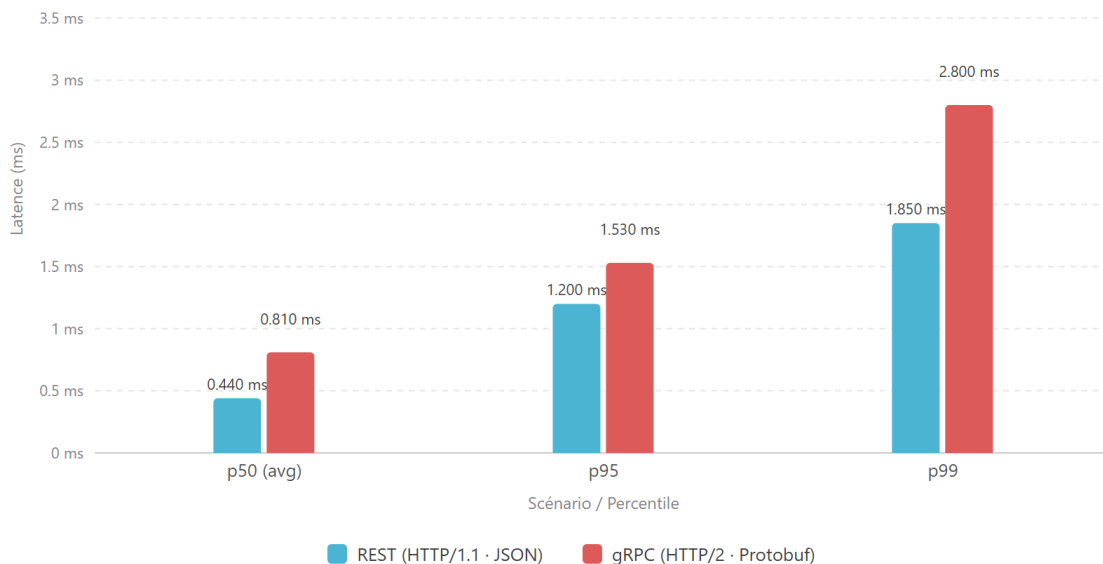
Sur les scénarios de lecture et d'écriture, REST obtient cependant de meilleurs résultats globaux dans notre environnement de test. Sur les scénarios de lecture, l'écart entre les deux protocoles est plus marqué qu'attendu.

REST atteint une latence moyenne de 0,41 ms pour un débit de 14 324 req/s, tandis que gRPC plafonne à 1,40 ms de moyenne et 5 241 req/s — soit environ 2,7 fois moins de requêtes par seconde pour une latence plus du triple. Ce n'est pas un léger avantage : en lecture unitaire à faible charge sur loopback, REST écrase gRPC.

Les percentiles racontent la même histoire. REST reste sous 1,26 ms au p95 et 2,24 ms au p99. gRPC monte déjà à 3,25 ms au p95 et 7,33 ms au p99, avec un max à 35 ms — un pic isolé, mais qui signale que le runtime gRPC a des moments de latence bien supérieurs à la médiane.



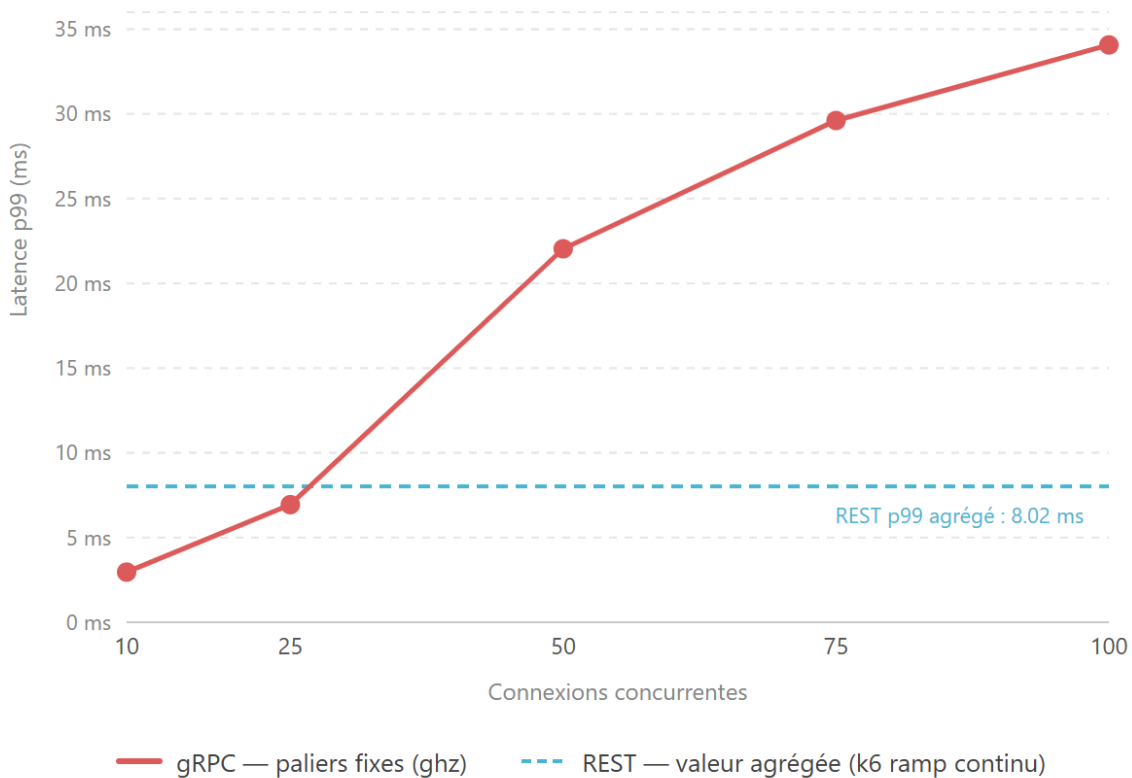
En écriture, REST conserve l'avantage mais l'écart se resserre. Moyenne à 0,44 ms pour 6 506 req/s côté REST, contre 0,81 ms et 4 087 req/s côté gRPC. Le delta RPS tombe à +59 % en faveur de REST — contre +173 % en lecture. Ce n'est pas anodin : quand le serveur fait un peu plus de travail (validation, écriture en store), le surcoût relatif du framing HTTP/2 pèse proportionnellement moins. gRPC se rapproche, mais reste en retrait.



Sur la charge progressive, la comparaison directe est à prendre avec des pincettes : k6 fait une montée continue de 10 à 100 VU sur environ 210 secondes, tandis que ghz ne sait pas faire de ramp natif — on a simulé ça avec cinq runs séparés à concurrence fixe (10, 25, 50, 75, 100). Les chiffres donnent un ordre de grandeur, pas un benchmark côte-à-côte strict.

Cela dit, les données sont parlantes. Côté REST, k6 a généré 7,85 millions de requêtes sur la durée du test, pour un débit moyen de 37 385 req/s, un p95 à 3,46 ms et un p99 à 8,02 ms. Zéro erreur sur l'ensemble du run. Côté gRPC, la montée en paliers donne une lecture plus fine de ce qui se passe quand la charge augmente :

Concurrence	RPS	avg	p95	p99
10	7 530	0,82 ms	1,86 ms	2,96 ms
25	10 600	1,48 ms	4,06 ms	6,94 ms
50	11 298	3,10 ms	9,63 ms	22,04 ms
75	16 613	3,05 ms	9,53 ms	29,61 ms
100	17 032	4,08 ms	13,23 ms	34,07 ms



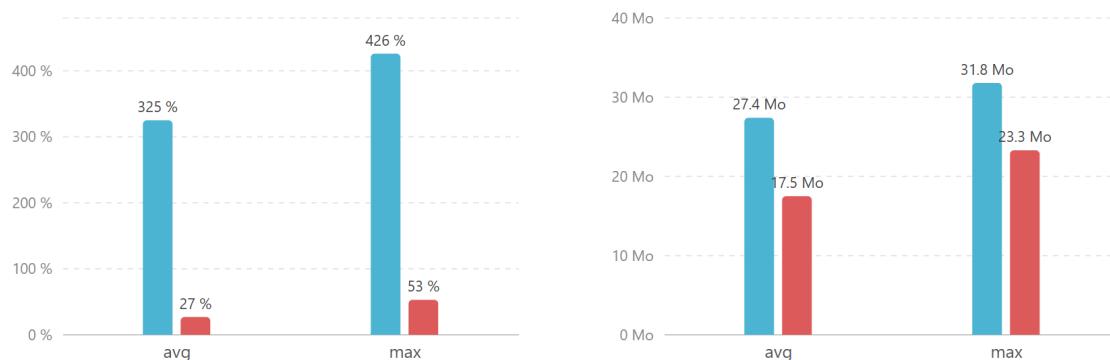
Le chiffre qui retient l'attention, c'est la p99 gRPC : elle passe de 2,96 ms à 10 connexions à 34,07 ms à 100 — un facteur 11. La médiane reste relativement stable, mais la queue de distribution explose. C'est le signe classique d'une file d'attente derrière les streams HTTP/2 multiplexés : tant que les connexions ne se marchent pas dessus tout va bien, mais dès que ça sature, certaines requêtes attendent longtemps. Pour SignalWatch avec des capteurs industriels et des timeouts potentiellement serrés, c'est un point d'attention SLO réel, pas juste une curiosité de benchmark.

On a également testé une variante REST avec compression gzip activée, pour voir si ça pouvait rapprocher REST de gRPC en taille de payload. Le résultat est contre-intuitif : REST+gzip est plus lent que REST nu, et à peine plus rapide que gRPC. Latence moyenne à 1,19 ms, p95 à 3,07 ms, débit à 6 074 req/s — contre 0,41 ms et 14 324 req/s sans compression.

L'explication est simple : sur un payload de 230 octets en loopback, le coût CPU de la compression dépasse largement le gain réseau — qui est de toute façon inexistant quand tout se passe sur la même machine. Gzip ne devient intéressant que quand le réseau est le goulot d'étranglement, c'est-à-dire en WAN avec une vraie latence. Dans notre environnement de test, l'activer ne fait qu'ajouter du travail inutile côté serveur.

Ce résultat a son importance pour la recommandation finale : gzip n'est pas une parade gratuite à la verbosité de JSON. On ne peut pas juste cocher "compression activée" et considérer que REST rivalise avec Protobuf sur la taille des échanges.

Pendant le scénario C, on a également relevé la consommation CPU et RAM des deux services via un utilitaire maison qui échantillonne les processus à 1 Hz. Les chiffres sont sans appel : sous forte charge, REST consomme en moyenne 325 % de CPU (pic à 426 %), contre 27 % en moyenne pour gRPC (pic à 53 %). Côté mémoire, REST tourne autour de 27,4 Mo de RAM contre 17,5 Mo pour gRPC — l'écart est moins spectaculaire, mais cohérent.



L'explication tient en grande partie au format. JSON doit être parsé et réencodé textuellement à chaque requête, sans réutilisation de contexte. Protobuf, lui, travaille en binaire avec des champs indexés par numéro — c'est mécaniquement moins cher à sérialiser. Le multiplexage HTTP/2 joue aussi : gRPC gère moins de connexions simultanées pour le même volume de requêtes, ce qui réduit la pression sur le scheduler.

Ce delta CPU n'est pas anodin à l'échelle SignalWatch. 10 000 événements par minute, 24h/24, 365 jours par an — une consommation processeur structurellement plus élevée côté REST se traduit directement en coût infra et en empreinte carbone. On y reviendra dans la section éco-conception.

Une nuance honnête s'impose cependant : les durées de capture ne sont pas symétriques. Le monitoring REST couvre 209 secondes de ramp continu, celui de gRPC couvre des paliers de quelques secondes chacun. Les ordres de grandeur sont fiables, mais une mesure à durée identique affinerait la comparaison.

Pourquoi REST gagne aussi nettement en latence absolue ici ? Trois facteurs se combinent. D'abord, le loopback annule complètement l'avantage taille de payload de gRPC — 135 octets contre 230, ça ne pèse rien quand il n'y a pas de réseau. Ensuite, l'overhead HTTP/2

(framing, métadonnées, négociation de stream) ne s'amortit pas sur des appels rares à faible concurrence : c'est un coût fixe que REST n'a pas. Enfin, les stacks HTTP/1.1 comme Gin sont taillées pour exactement ce type de scénario — requêtes courtes, connexions légères, parsing JSON très optimisé.

Ça ne veut pas dire que gRPC perd sur toute la ligne. Son payload binaire reste ~40 % plus compact, et c'est le seul avantage qui tient même en loopback — quand le réseau redevient le goulot d'étranglement en WAN, cet écart pèse. Surtout, les mesures CPU le disent clairement : gRPC est structurellement plus économe côté serveur, avec un tiers de la consommation processeur de REST sous forte charge et une empreinte mémoire inférieure. Son RPS monte aussi de façon plus régulière en montée de charge, ce que les paliers du scénario C confirment.

Là où gRPC pose un problème concret, c'est sur la p99 au-delà de 50 connexions concurrentes. Une latence de queue qui décuple entre 10 et 100 connexions, c'est un risque réel pour des capteurs industriels avec des timeouts serrés — pas un artefact de benchmark. À ça s'ajoute une couche de complexité opérationnelle non négligeable : toolchain Protobuf, fichiers .proto à maintenir, debug d'un format binaire qui ne se lit pas à l'œil nu. Des éléments qui comptent quand on évalue la maintenabilité à long terme d'une stack.

Plan de veille :

1. Objectif de la veille
2. Répartition des thèmes
3. Source par thème
4. Planning de la semaine
5. Format de restitution interne
6. RACI des livrables

Objectif de la veille

La veille doit alimenter trois dimensions du rapport de veille :

La dimension technique pour comprendre en profondeur REST, gRPC, GraphQL et les messages Brokers.

La dimension éco-conception avec une intégration Green IT pour justifier la recommandation finale.

La dimension réglementaire niveau RGPD.

Répartition des thèmes

On a réparti les thèmes de manière à ce que chacun d'entre nous soit expert de son protocole.

Yanis : gRPC / Protobuf / HTTP2 / Message Brokers et éco-conception.

Arthur : REST / HTTP / GraphQL / RGPD / CNIL

Source par thème

Source Yanis :

- gRPC : Documentation officielle grpc.io, blog CNCF (cncf.io/blog)
- Protobuf : protobuf.dev, guide Buf (buf.build/docs)
- HTTP/2 : RFC 7540, RFC 9113 (HTTP/2 révisé 2022)

- Message brokers : Documentation officielle RabbitMQ (rabbitmq.com), documentation officielle Kafka (kafka.apache.org)
- Eco conception : RGEN (ecoresponsable.numerique.gouv.fr) , Green IT (greenit.fr)

Source Arthur :

- REST/HTTP : RFC 9110, MDN Web docs (developer.mozilla.org/en-US/docs/Web/HTTP),
- GraphQL : Documentation officielle (graphql.org/learn), GraphQL blog (apollographql.com/blog) , howtographql.com
- RGPD / CNIL : Site officiel CNIL (cnil.fr), texte du rgpd (eur-lex.europa.eu) , guide de la CNIL pour les devs (github.com/LINCnil/Guide-RGPD-du-developpeur)

Critères de fiabilité

On n'a pas traité toutes les sources de la même façon. La priorité va aux sources officielles — RFC, documentations éditeurs — devant les publications académiques (IEEE, ACM), elles-mêmes devant les blogs d'experts reconnus comme ceux de la CNCF ou de Cloudflare. Les forums et discussions StackOverflow ne sont jamais cités comme source primaire.

Sur la récence, on a privilégié des sources de moins de 3 ans, sauf pour les RFC fondatrices comme la RFC 7540 (HTTP/2) ou la RFC 9110 (HTTP sémantique) — leur ancienneté ne les rend pas caduques, elles restent la référence.

Tout chiffre repris dans le rapport doit apparaître dans au moins deux sources indépendantes. Quand ce n'est pas le cas, on le signale explicitement plutôt que de faire semblant.

Enfin, on a lu avec recul les benchmarks publiés par un éditeur sur sa propre technologie — une étude Google qui prouve que gRPC bat REST, ça mérite qu'on vérifie les conditions de test avant de citer quoi que ce soit.

Planning de la semaine

Jour	Arthur (REST)	Yanis (gRPC)	Veille (fil rouge)
J1	Cadrage à deux · init projet Go · début service REST	Cadrage à deux · lecture doc Protobuf · début fichier .proto	Arthur : REST/HTTP · Yanis : gRPC/Protobuf
J2	Fin service REST + tests curl	Code gRPC + implémentation + tests grpcurl	Arthur : REST (fin) · Yanis : HTTP/2
J3	Vérif équivalence fonctionnelle (ensemble) · scénarios bench A et B côté REST	Vérif équivalence fonctionnelle (ensemble) · scénarios bench A et B côté gRPC	Arthur : GraphQL · Yanis : Kafka/RabbitMQ
J4	Scénario C + runs + relevés CPU/RAM (ensemble) · script Python graphes	Idem	Arthur : RGPD/CNIL · Yanis : RGENS/Green IT
J5	Rédaction sections A, B1, B3 du rapport	Rédaction sections B2, B4, D du rapport	Finalisation bibliographie
J6	Veille élargie : protocoles émergents (HTTP/3, MQTT) pour la section "limites"	Idem	—
J7	Bibliographie finale : formatage des références, vérification des liens morts	Idem	—

Restitution interne :

On travaille sur le même Google Doc tout au long de la semaine. Chaque soir vers 18h, on se fait un point rapide de 10 minutes autour de trois questions : qu'est-ce qui est fait,

qu'est-ce qui bloque, et quelle est la prochaine action. L'idée c'est de garder ça court — pas une réunion, juste une synchro.

Pour la veille, chaque source lue donne lieu à une fiche courte dans un doc "biblio brute" commun : titre, lien, et trois points clés à retenir. Ça évite de relire cinq articles la veille du rendu pour se souvenir de ce qu'on en avait tiré.

Côté git, on commit souvent et avec des préfixes explicites : **feat** : pour du code, **bench** : pour les scripts et résultats, **docs** : pour le rapport et le plan de veille. Comme ça l'historique reste lisible sans avoir à ouvrir chaque diff.

Risques et plan B

On a identifié quatre risques principaux sur le projet. Le premier c'est la variabilité des résultats de benchmark — les mesures en local peuvent fluctuer selon les processus qui tournent en parallèle. Pour y remédier, on fait minimum trois runs par scénario et on retient la médiane plutôt que la moyenne.

Le deuxième risque c'est un désaccord sur la recommandation finale. C'est peu probable vu qu'on a une matrice de décision pondérée, mais si ça arrive, on la tranche ensemble à J6 avant de rédiger la section F.

Troisième risque : le retard sur la rédaction. Pour l'éviter, on rédige les sections au fil de l'eau dès J3, pas en sprint final à J5-J6 quand on est déjà fatigués.

Dernier risque, plus technique : une source citée dans le rapport qui devient indisponible entre le moment où on la lit et le rendu. On archive systématiquement via web.archive.org au moment de la lecture pour avoir une copie stable.

RACI des livrables

Livable	Arthur	Yanis
Service REST (Gin)	R, A	C
Service gRPC (grpc-go)	C	R, A
Modèle Sensor commun & store in-memory	R, A	R
Scripts de benchmark k6 — scénarios A, B, C (REST)	R, A	C
Scripts de benchmark k6 — scénarios A, B, C (gRPC)	C	R, A
Collecte des résultats bruts (CSV/JSON)	R, A	R
Relevés CPU/RAM pendant benchmark	R	R, A
Script Python de génération de graphes	C	R, A
Rapport — Section A (méthodologie de veille)	R, A	C
Rapport — Section B1 (REST) & B3 (GraphQL)	R, A	C
Rapport — Section B2 (gRPC) & B4 (message brokers)	C	R, A
Rapport — Section C (résultats benchmark)	R, A	R
Rapport — Section D (éco-conception)	C	R, A
Rapport — Section E (RGPD)	R, A	C
Rapport — Section F (recommandation & matrice)	R	R, A
Support de présentation (10 slides)	R	R, A
README.md principal du dépôt	R, A	C
Plan de veille (ce document)	R, A	C
Vérification finale de la structure du dépôt	R, A	R